Keywords: agentic video • world-state memory • cross-shot consistency • video generation

Any Time, Any Space

A World-State-Centric Framework for Agentic Video Generation Maintains Two Persistent Memories—Scene Geometry and Indexed Dynamic Events—to Ensure Full Spatio-Temporal Coherence Across Nonconsecutive Shots

By Weijie Wang, Haomin Zhang, Bohan Zhuang et al.

arXiv:2610.20571 • October 2026

Long-form narrative video generation requires that any shot—requested at any time, depicting any space—remain spatially and temporally consistent with every other shot in the production. ATAS (*Any Time, Any Space*) achieves this by maintaining a *scene memory* of registered static geometry and an *event memory* of dynamic observations indexed by scene, event, and time. Before generating each shot the system retrieves the relevant evidence; after generation a review gate accepts or rejects the clip before registering it into memory. Across four video generators ATAS improves every reported cross-shot consistency metric and tops all six human-rating criteria.

AT A GLANCE

Problem: Long-form agentic video suffers from location drift and temporal contradictions when shots are generated nonconsecutively; existing methods lack a shared spatial and temporal state.

Why it matters: Agentic video is an emerging AI application where coherence failures break narrative trust; no prior approach solves multi-shot spatio-temporal consistency.

Method: Scene memory stores registered geometry per location; event memory stores dynamic observations indexed by scene, event, and time; a review gate decouples film delivery from memory admission.

Result: Highest aggregate benchmark score per backbone; tops all six human-rating criteria across four video generators.

Evidence: Four video generators; spatial revisit, event recapture, and temporal consistency benchmarks; human evaluation on six criteria.

The Big Picture

Narrative video pipelines built from large generative models suffer from *scene drift*: ask the same model to re-enter a room from a new camera angle three shots later, and it will produce a room that contradicts the layout established earlier. The same problem afflicts event recapture—a flashback to an event the viewer already witnessed will not match the original.

These failures arise because every shot is generated independently. The model conditions on a text prompt and, optionally, the most recent frame, but neither source encodes the durable spatial and temporal facts the narrative has established. ATAS resolves this by replacing the stream of frames with a *world-state representation*: two structured memories that persist across the entire production, regardless of generation order.

Why Existing Approaches Fall Short

Prompt conditioning passes shot descriptions between generation calls. It provides no spatial grounding; the model must re-invent geometry from text at each step, and two independent descriptions of the same room will produce inconsistent geometry.

Recent-frame conditioning threads the previous clip's last frame into the current generation. This provides visual continuity for adjacent shots, but cannot support true out-of-order revisits or event recall across large temporal distances in the screenplay.

3D-guided methods impose rigid scene constraints extracted from monocular depth or NeRF reconstruction. These are difficult to update as the narrative introduces new objects, character motion, and destructive events; they also require per-production reconstruction passes that scale poorly.

Agent planning without state improves shot ordering and prompt engineering but leaves individual generation calls unmodified; without a persistent world model, cross-shot coherence is not guaranteed.

Core Mechanism: ATAS

ATAS structures the agentic loop around two memories:

Scene Memory stores a geometric descriptor $G(s_i)$ for each named scene s_i. When a location is first visited, the generated clip is reconstructed and the geometry registered. All subsequent shots at s_i retrieve $G(s_i)$ and condition on it, ensuring consistent spatial layout regardless of camera angle or narrative time.

Event Memory stores a clip descriptor $D(s_i, e_j, t)$ for each combination of scene s_i, event e_j, and time t. When an action or state change is generated, the resulting clip is reconstructed and registered at the appropriate indices. A later shot requesting a replay retrieves the stored entry rather than re-generating from a text description.

Review Gate: Before registering any clip, ATAS applies quality and state-consistency checks. A rejected clip is regenerated; film delivery is independent of memory admis-

sion, so the output stream never blocks and hallucinated geometry is never registered.

Technical Formulation

Let $\mathcal{S} = \{s_1, \ldots, s_N\}$ be named scenes and $\mathcal{E} = \{e_1, \ldots, e_M\}$ be narrative events. Scene memory maps each scene to its registered geometry:

$$G(s_i) = \text{Reconstruct}\Big(v^{(1)}(s_i)\Big)$$

where $v^{(1)}(s_i)$ is the first accepted clip at scene s_i.

Event memory maps a triple to a dynamic descriptor:

$$D : \mathcal{S} \times \mathcal{E} \times \mathbb{R}_{\geq 0} \to \mathcal{D}$$

For a shot request $r = (s_i, e_j, t, \text{cam})$, the conditioning evidence is the concatenation of the two retrieved descriptors:

$$C(r) = G(s_i) \oplus D(s_i, e_j, t)$$

and the generator produces:

$$v_r = \text{VideoGen}(r,\ C(r))$$

After the review gate accepts v_r, the event memory is updated:

$$D(s_i, e_j, t) \ \leftarrow\ \text{Reconstruct}(v_r)$$

If the review gate rejects v_r, the clip is regenerated; the memory state is unchanged, preventing error propagation.

Learning or Inference Procedure

1. Parse the screenplay into shot requests $r = (s_i, e_j, t, \text{cam})$.
2. For each request, query scene memory for $G(s_i)$ and event memory for $D(s_i, e_j, t)$; retrieve empty descriptors for unseen entries.
3. Condition the video generator on $C(r)$ and generate clip v_r.
4. Apply the review gate; if accepted, update $D(s_i, e_j, t)$ and $G(s_i)$ (on first visit); deliver v_r to the output stream.
5. If rejected, return to step 3 with a refreshed generation seed.
6. Repeat until all shots are delivered.

No additional model training is required. ATAS is a retrieval-and-conditioning framework that wraps any video generation backbone.

What the Guarantee Says

ATAS does not make formal guarantees on per-shot quality, which depends on the underlying generator. Its empirical claims are:

- Every reported cross-shot consistency metric improves across all four tested video generators when ATAS is added.
- ATAS achieves the highest aggregate benchmark score per backbone on the spatial-revisit and event-recapture tasks.
- Human raters score ATAS highest on all six criteria (narrative coherence, spatial accuracy, temporal consistency, visual quality, event fidelity, audio-visual alignment) across all tested generators.

Experimental Findings

- **Backbones:** Four video generators covering diffusion-based and autoregressive architectures.
- **Spatial revisit:** ATAS outperforms compared agent-based and 3D-guided methods on the target-image spatial revisit task.
- **Event recapture:** ATAS leads on three event-recapture criteria; prior methods degrade to text-driven re-generation.
- **Temporal consistency:** All temporal-consistency metrics improve under ATAS for every backbone.
- **Human evaluation:** ATAS tops all six human-rating criteria across all tested generators; previous methods trail on at least two.
- **Out-of-order generation:** ATAS maintains consistency even when shots are generated out of screenplay order.

Ablations and Interpretation

Without scene memory: Spatial layout drifts across revisit shots; objects disappear or change position; the spatial-revisit score drops sharply.

Without event memory: Event recapture degrades to text-driven re-generation; temporal positions shift; event-fidelity scores drop.

Without review gate: Hallucinated geometry is registered and propagates to subsequent shots; errors compound as the production grows.

Memory granularity: Finer time indexing in event memory yields better event recapture; coarser indexing reduces memory overhead at the cost of temporal precision.

Why retrieval beats prompting: Geometry extracted from accepted clips encodes exact object positions that a text description cannot; conditioning on this evidence directly constrains the generator's spatial output distribution, while text descriptions leave spatial degrees of freedom underdetermined.

Reference

Weijie Wang, Haomin Zhang, Bohan Zhuang, et al. **Any Time, Any Space: A World-State-Centric Framework for Agentic Video Generation**. arXiv:2610.20571, October 2026.
https://arxiv.org/abs/2610.20571

Keywords: game agents • reinforcement learning • action space • synthetic environments

Player-1: Learning to Play from Synthetic Experience

Extending the VLM Action Space with Relative Motion, Sustained Inputs, and Composite Actions, then Training with PPO in 16 Synthetic Environments, Triples MineDojo-300 Success and Transfers to Held-Out Games

By Mingyu Ouyang, Yiran Wang, Siyuan Hu et al. (NUS, Tencent ARC/IEG, NTU)

arXiv:2610.20584 • October 2026

Standard computer-use agents fail at digital gameplay not because their policies are too weak, but because their action spaces cannot express the motor primitives games require: relative pointer motion, sustained directional inputs, and composite simultaneous key combinations. *Player-1* fixes the representation gap first: it extends the GUI action vocabulary, fine-tunes on 6,044 curated gameplay trajectories (SFT), then runs PPO in Player-1-Gym—16 synthetic 2D and 3D environments spanning five control families. Player-1-9B raises MineDojo-300 success from 12.63% (best prior baseline) to **38.33%**; gains transfer to ViZDoom-10 and GameWorld without any game-specific fine-tuning.

AT A GLANCE

Problem: Standard GUI action spaces lack relative pointer motion, sustained inputs, and composite key combos—the primitives digital games require; agents fail not from limited policy but from limited vocabulary.

Why it matters: Game-playing is a rich testbed for continuous motor-skill learning in VLMs; solving the action-space gap is a prerequisite for any game-capable generalist agent.

Method: Extend the action space; SFT on 6,044 curated trajectories; PPO in Player-1-Gym (16 procedurally generated 2D/3D environments, 5 control families: aiming, combat, driving, navigation, UI).

Result: Player-1-9B: 38.33% on MineDojo-300 (+25.7 pp over best prior); 35B-A3B reaches 41.04%; transfer to held-out ViZDoom-10 and GameWorld.

Evidence: MineDojo-300, ViZDoom-10, GameWorld; 9B and 35B-A3B; SFT-only ablation.

The Big Picture

Vision-language models now navigate browsers, fill forms, and operate desktop applications with reasonable accuracy. Interactive games present a qualitatively different challenge: staying alive in a first-person shooter, steering a vehicle, executing a combat combo, or sprinting while turning requires motor skills— sustained, feedback-driven, multi-axis control—that desktop GUI tasks never demand.

The failure mode is not a policy problem. If the action space cannot express "move the mouse 12 pixels right while holding the left mouse button," no amount of policy optimization will produce that action. Player-1 diagnoses this as a representation gap and closes it before applying RL.

Why Existing Approaches Fall Short

SFT-only agents can imitate successful play trajectories, but imitation is brittle to novel game states and does not generalize to games outside the demonstration distribution.

Standard GUI agents treat games like web navigation: they produce discrete, stateless click-and-keystroke actions. They cannot execute the sustained directional inputs needed for driving, the relative pointer sweeps needed for aiming, or the simultaneous multi-key presses needed for combat.

Game-specific RL agents achieve strong results on individual titles (Atari, StarCraft) but require game-specific reward shaping, environment wrappers, and action-space engineering; they do not transfer to new games or VLM-based GUI architectures.

Real-game RL training requires running live game executables: it is slow, platform-dependent, and legally constrained for commercial titles.

Core Mechanism: Extended Action Space and Player-1-Gym

Extended action space:

$$\mathcal{A}_{\text{ext}} = \mathcal{A}_{\text{std}} \cup \mathcal{A}_{\text{rel}} \cup \mathcal{A}_{\text{hold}} \cup \mathcal{A}_{\text{combo}}$$

where $\mathcal{A}_{\text{rel}}$ adds relative pointer motions $(\Delta x, \Delta y) \in \mathbb{R}^2$, $\mathcal{A}_{\text{hold}}$ adds hold/release primitives for keys and buttons, and $\mathcal{A}_{\text{combo}}$ adds composite simultaneous key-plus-pointer sequences.

Player-1-Gym is a suite of 16 synthetic 2D and 3D environments covering five control families:

1. Aiming (first-person pointer control)
2. Combat (sustained + combo actions under adversarial pressure)
3. Driving (continuous relative steering)
4. Navigation (spatial orientation + locomotion)

5. UI interaction (menus, inventory, crafting)

Environments are procedurally generated so each episode presents a novel layout, preventing simple memorization of visual patterns.

Technical Formulation

The SFT phase minimizes the standard next-token prediction loss over a demonstration dataset $\mathcal{D} = \{(o_t, a_t)\}_{t=1}^{T}$ annotated with extended actions:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(o_t,a_t)\sim\mathcal{D}}[\log \pi_\theta(a_t \mid o_t)]$$

The PPO phase optimizes the clipped surrogate objective with advantage estimates computed via GAE:

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1{-}\varepsilon, 1{+}\varepsilon)\,\hat{A}_t\Big)\Big]$$

where the probability ratio is $r_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\theta_{\text{old}}}(a_t \mid o_t)$ and $\hat{A}_t$ is the generalized advantage estimate.

The value function $V_\phi(o_t)$ is trained simultaneously with

$$\mathcal{L}_V(\phi) = \mathbb{E}_t\left[\left(V_\phi(o_t) - \hat{R}_t\right)^2\right]$$

where $\hat{R}_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ is the discounted return.

Reward signals for Player-1-Gym are binary task-success indicators plus progress-based shaped rewards derived from task predicates.

Learning or Inference Procedure

1. **SFT:** Fine-tune the VLM on 6,044 human gameplay trajectories annotated with $\mathcal{A}_{\text{ext}}$ labels.
2. **Deploy to Player-1-Gym:** Instantiate 16 environments with procedurally generated episodes; set episode timeout.
3. **PPO rollout:** Asynchronous workers generate episodes; a central policy-update server applies gradient steps every N rollout steps.
4. **Evaluation:** Zero-shot inference on MineDojo-300, ViZDoom-10, and GameWorld using the RL-trained policy; no game-specific fine-tuning.

What the Guarantee Says

Player-1 establishes the following empirical claims across model sizes and held-out benchmarks:

- Player-1-9B: MineDojo-300 success = 38.33% vs. 12.63% for the best prior baseline (+25.7 pp).
- Player-1-35B-A3B: MineDojo-300 success = 41.04% (+17.7 pp over its own SFT policy).
- Zero-shot transfer to ViZDoom-10 and GameWorld (both outside Player-1-Gym); both show positive gains over the SFT-only policy.
- SFT-only policy scores substantially below the PPO-trained model on all evaluated benchmarks, confirming RL drives the improvement.

Experimental Findings

- **MineDojo-300:** Player-1-9B: 38.33%; Player-1-35B-A3B: 41.04%; best prior: 12.63%.
- **ViZDoom-10:** Positive transfer; largest gains on aiming and combat subtasks that most require relative-motion actions.
- **GameWorld:** Positive transfer across navigation and UI subtasks.
- **SFT ablation:** SFT-only policy (no PPO) shows substantial gap below the full Player-1 model on all three benchmarks.
- **Control family breakdown:** Aiming and combat show the largest absolute gains, confirming that the new action primitives are the key bottleneck for those task families.

Ablations and Interpretation

Action space ablation: Removing $\mathcal{A}_{\text{rel}}$ from the action set causes a large drop on aiming and driving tasks; removing $\mathcal{A}_{\text{combo}}$ degrades combat performance.

Gym environment count: Performance on MineDojo-300 improves as the number of Player-1-Gym environments increases; the diversity of motor tasks covered—not just quantity—determines transfer breadth.

SFT warmup importance: Removing the SFT phase and training RL from the base VLM significantly slows convergence; the SFT demonstration data provides a useful initial policy distribution that shortens RL exploration.

Model scale: 35B-A3B outperforms 9B (+2.71 pp on MineDojo-300); the gap is larger on ViZDoom, suggesting complex first-person reasoning benefits more from scale than pure motor-control tasks.

Why transfer works: The five control families in Player-1-Gym cover the motor primitives common to most PC games; RL-acquired motor policies generalize across visual appearances and reward structures that differ from training, because the underlying motor sub-problem is shared.

Reference

Mingyu Ouyang, Yiran Wang, Siyuan Hu, et al. **Player-1: Learning to Play from Synthetic Experience**. arXiv:2610.20584, October 2026.
https://arxiv.org/abs/2610.20584

Keywords: latent reasoning • hybrid agents • tool use • token efficiency

Hybrid Reasoning Agents

Switching Between Latent-Space Computation and Text Generation Delivers +10.4% Accuracy, 35–55% Fewer Tokens, and 2.7–3.7× Lower Cost Across 11 Benchmarks

By Jiaru Zou, Yunzhe Qi, Sirui Chen, Federico Bianchi, Shang Zhu, Pan Lu, Jingrui He & James Zou

arXiv:2610.18861 • October 2026

Text-based agents decode every reasoning step as expensive tokens, yet latent-only models cannot natively call tools or produce interpretable traces. *HybridAgent* resolves the dilemma by switching between latent-space computation and text generation within a single generation pass via a lightweight **HybridLink** module, while a **continuous working memory** preserves hybrid context across turns without lossy summarization. Evaluated on Qwen3-Base at 4B, 8B, and 14B across 11 benchmarks, HybridAgent achieves a +10.4% accuracy gain, 35.5–54.7% fewer tokens, and 2.7–3.7× lower cost compared to text-only and latent-only baselines.

AT A GLANCE

Problem: Text agents decode all reasoning as tokens (expensive); latent-only models cannot call tools or produce inspectable reasoning.

Why it matters: Inference compute is the frontier bottleneck; hybrid modes offer the efficiency of latent computation with the expressiveness of text-based tool use.

Method: HybridLink transitions between text and latent modes; working memory preserves hybrid context; HybridSFT labels switch points; HybridRL jointly penalizes errors, excess tokens, and tool failures.

Result: +10.4% avg accuracy; 35.5–54.7% fewer tokens; 2.7–3.7× lower cost than text-only baselines.

Evidence: Qwen3-Base at 4B, 8B, and 14B; 11 benchmarks covering math, science, medical reasoning, and tool-use tasks.

The Big Picture

Current agent paradigms force a binary choice: text-based interaction is interpretable and natively supports tool calls, but materialises every reasoning step as decoded tokens, inflating latency and cost. Latent-only models process internally without decoding intermediate states, but they cannot call APIs or external databases and their reasoning is inaccessible to users or auditors.

HybridAgent dissolves the tradeoff. The key observation is that internal reasoning—planning, hypothesis formation, chain-of-thought—does not need to be decoded at every step. Only when the model must interact with the environment (call a tool, present an answer, request input) does it need to produce text. A principled switch mechanism at the right moments exploits latent efficiency while preserving text expressiveness.

Why Existing Approaches Fall Short

Text-based agents decode all intermediate reasoning, producing interpretable but expensive token streams; context length grows with reasoning depth.

Latent-only models compute internally without materialising tokens; efficient but unable to call external tools; reasoning is inaccessible to downstream users or auditors.

Shallow fusion approaches interleave text and embedding layers without a principled switch mechanism; they suffer from mode confusion at transition points, degrading both the coherence of latent computation and the expressiveness of text interaction.

Core Mechanism: HybridLink and Working Memory

Two components distinguish HybridAgent from prior fusion approaches:

HybridLink is a lightweight adapter inserted at a designated layer l of the base transformer. It handles transitions in both directions: text-to-latent (compressing recent text context into an initial latent state vector) and latent-to-text (projecting the latent state back into the token embedding space at tool-call or answer generation points).

Continuous working memory is a fixed-size buffer of latent vectors that persists across turns. Unlike KV-cache compression, which discards information to save memory, the working memory preserves the full hybrid context at constant cost via a recency-weighted update rule.

Post-training uses two stages: **HybridSFT** fine-tunes on trajectories where ground-truth switch points are labelled; **HybridRL** optimises the full model with a joint reward that penalises incorrect answers, excessive text decoding, and tool-call failures.

Technical Formulation

Let the transformer have L layers and hidden dimension d. At turn t the model maintains a text prefix $x_t \in \mathcal{V}^*$ and a latent state $z_t \in \mathbb{R}^{d_z}$. HybridLink at layer l computes:

$$z_t = \text{HybridLink}^{(\text{t}\to\text{l})}(h_l(x_t),\, z_{t-1})$$

$$x'_t = \text{HybridLink}^{(\text{l}\to\text{t})}(z_t)$$

where $h_l(x_t)$ is the hidden state at layer l for the text prefix, the first equation handles text-to-latent transitions, and the second handles latent-to-text transitions at tool-call points.

The working memory update is:

$$z_t = (1 - \alpha)\, z_{t-1} + \alpha \cdot f(z_{t-1}, h_L(x_t))$$

with α a learned gate scalar and f a two-layer MLP.

The HybridRL objective is:

$$\mathcal{J}(\theta) = \mathbb{E}[R_{\text{task}} - \lambda_1 \|x_t^{\text{reason}}\|_1 - \lambda_2 \cdot \mathbf{1}[\text{tool fail}]]$$

where $\|x_t^{\text{reason}}\|_1$ counts decoded reasoning tokens and λ_1, λ_2 are regularisation weights.

Learning or Inference Procedure

1. **Label switch points** in existing reasoning trajectories: text-to-latent at the start of each reasoning block; latent-to-text before each tool call or final answer.
2. **HybridSFT:** Supervised fine-tuning on labelled trajectories, training HybridLink to transition at the correct points.
3. **HybridRL:** Reinforcement fine-tuning with the joint reward $\mathcal{J}(\theta)$, sharpening the autonomous switch policy.
4. At **inference**: the model operates without human guidance; when reasoning continues, it stays in latent mode; when a tool call or answer is needed, it switches to text mode via HybridLink.

What the Guarantee Says

Across all three Qwen3-Base model scales (4B, 8B, 14B) and all 11 evaluated benchmarks:

- Average accuracy gain over text-only and latent-only baselines: **+10.4**%.
- Token reduction versus text-only baseline: **35.5–54.7%**.
- Cost reduction: **2.7–3.7**× lower at matched accuracy.

The relative advantage grows slightly with model scale, suggesting that larger models use the latent mode more judiciously.

Experimental Findings

- **Models:** Qwen3-Base at 4B, 8B, and 14B parameters.
- **Benchmarks (11):** Mathematics, science, and medical reasoning domains; tool-use tasks requiring search and API calls.
- +10.4% average accuracy gain; highest gains on multi-step tool-use tasks that benefit most from latent-mode efficiency.
- Latent-only models cannot complete tool-dependent tasks; HybridAgent's gain over them is substantially larger than its gain over text-only agents.
- Token counts: 35.5–54.7% below text-based chains of equal or lower accuracy.
- Cost: 2.7–3.7× reduction based on token count × per-token price.

Ablations and Interpretation

Without HybridSFT (RL only): Convergence slows; final accuracy is lower; supervised switch-point labelling provides a critical warm-start.

Without working memory: Accuracy drops on multi-turn tasks requiring retention of earlier tool outputs; the model fails to carry latent context across conversation turns.

Without latent mode (text-only baseline): Token count rises and accuracy falls below HybridAgent on all 11 benchmarks.

HybridLink layer placement: Inserting at middle layers ($l \approx L/2$) outperforms early and late placement; early placement loses text context before reasoning begins, late placement provides insufficient latent depth for meaningful computation.

Why it works: The model learns that tool-call latency and token cost dominate total inference time; switching to latent mode for the reasoning portion that does not need to be observed is the "free efficiency" move, and HybridRL reinforces this behaviour.

Reference

Jiaru Zou, Yunzhe Qi, Sirui Chen, Federico Bianchi, Shang Zhu, Pan Lu, Jingrui He, James Zou. **Hybrid Reasoning Agents**. arXiv:2610.18861, October 2026.
https://arxiv.org/abs/2610.18861

Keywords: video benchmark • omni-modal • temporal grounding • audio-visual understanding

OmniCompass: A Holistic Benchmark for Omni-Modal Video Grounding and Understanding

Paired-Setting Evaluation of 50 MLLMs Reveals Widespread Pattern-Matching and Persistent Grounding Failures in Both Vision-Language and Audio-Visual Tracks

By Xuyang Liu, Shihao Wang, Xinyu Wei, Minghan Li, Zeqing Wang, Zuhao Yang, Zichen Wen & Lei Zhang

arXiv:2610.18829 • October 2026

Today's video benchmarks test language or vision in isolation, leaving the joint audio-visual grounding capabilities of omni-modal models unmeasured. *OmniCompass* introduces a paired evaluation protocol—each instance evaluated under text-only *and* text+evidence conditions—across two tracks: OmniCompass-VL (820 videos, 2,460 instances) and OmniCompass-AV (240 videos, 720 instances from narrative-rich, audio-dense content). Evaluation of 50 MLLMs finds that open-source models trail proprietary ones by 15–20 points on VL tasks and more on AV tasks; widespread paired-setting inconsistency (>30% of instances) exposes surface-level pattern matching in even the best-performing models.

AT A GLANCE

Problem: Video benchmarks test language or vision in isolation; no systematic measurement of joint audio-visual grounding exists.

Why it matters: Omni-modal models are deployed in video understanding; benchmarks must reveal whether they genuinely ground queries in evidence or pattern-match instruction formats.

Method: Two-track benchmark with paired evaluation (text-only/text+evidence); 8 VL task categories and 5 AV categories; temporal grounding, spatial grounding, and QA subtasks.

Result: 50 MLLMs tested; open-source trails proprietary by 15–20 pp on VL and more on AV; >30% paired-setting inconsistency in the majority of models.

Evidence: 820+240 videos, 2,460+720 instances; inter-annotator $\kappa > 0.8$.

The Big Picture

Omni-modal models can now process video, audio, and text simultaneously, yet their evaluation lags: benchmarks still test modalities in isolation. A model that answers a temporal grounding question correctly under text-only conditions may fail when the same question is accompanied by contradictory visual evidence—or vice versa. This mismatch is invisible to single-condition evaluation.

OmniCompass is built around a paired evaluation protocol that tests every instance twice: once with text alone and once with the relevant visual or audio evidence. A model that genuinely understands the target should respond consistently (and correctly) under both conditions. Inconsistency across conditions is a diagnostic signal: the model is pattern-matching the instruction format rather than grounding in the evidence.

Why Existing Approaches Fall Short

Standard VQA benchmarks test single-image comprehension; they do not cover video-level temporal or spatial grounding across a narrative.

Text-only LLM benchmarks ignore visual and audio modalities entirely; they cannot measure multimodal grounding by construction.

Existing video QA benchmarks (MVBench, VideoMME, etc.) focus on visual understanding without audio-dependent tasks or the paired-setting probe that would reveal instruction-format reliance.

Prior audio-visual benchmarks are small-scale and lack the structured temporal/spatial annotation and the paired consistency diagnostic that OmniCompass provides.

Core Mechanism: Paired Evaluation Design

Two evaluation tracks:

OmniCompass-VL (Vision-Language): 820 videos, 2,460 instances. Questions require visual evidence. Eight task categories: five perception (object recognition, action recognition, attribute identification, counting, OCR) and three reasoning (causal, commonsense, temporal reasoning). Three subtask types: temporal grounding (TG), spatial grounding (SG), and question answering (QA).

OmniCompass-AV (Audio-Visual): 240 videos, 720 instances from narrative-rich, audio-dense content. Questions require joint audio-visual evidence. Five task categories: three perception (speech recognition, sound identification, speaker localization) and two reasoning (audio-visual causal reasoning, event coordination). Same TG/SG/QA subtask structure.

Paired evaluation: Each instance is evaluated twice per model: text-only and text+evidence. Inconsistency

between conditions flags surface-level pattern matching.

Technical Formulation

For model $\mathcal{M}$, query q_i, visual/audio evidence v_i, and ground-truth answer a_i^*, define per-instance scores:

$$s_i^{\text{text}} = \mathbf{1}[\mathcal{M}(q_i) = a_i^*], \quad s_i^{\text{multi}} = \mathbf{1}[\mathcal{M}(q_i, v_i) = a_i^*]$$

The **consistency score** across N instances is:

$$C(\mathcal{M}) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left[s_i^{\text{text}} = s_i^{\text{multi}}\right]$$

A high $C(\mathcal{M})$ with high $\sum_i s_i^{\text{multi}}$ indicates reliable grounding; a high $C(\mathcal{M})$ with low accuracy indicates consistent failure; a low $C(\mathcal{M})$ regardless of accuracy indicates format sensitivity.

Grounding precision is measured separately for temporal and spatial tasks. For a predicted interval $[\hat{t}_s, \hat{t}_e]$ and ground-truth $[t_s, t_e]$:

$$\text{T-IoU} = \frac{[\hat{t}_s, \hat{t}_e] \cap [t_s, t_e]}{[\hat{t}_s, \hat{t}_e] \cup [t_s, t_e]}$$

Spatial grounding uses standard bounding-box IoU.

Learning or Inference Procedure

OmniCompass is a benchmark; no model training is involved. Evaluation protocol:

1. Collect videos with expert-annotated temporal segments, spatial bounding boxes, and QA pairs (inter-annotator $\kappa > 0.8$).
2. Adversarially filter questions solvable from text alone.
3. Construct paired instances: text-only and text+evidence versions of every question.
4. Evaluate each model in zero-shot mode on all instances.
5. Compute per-track accuracy, T-IoU, spatial IoU, and consistency scores; stratify by task category and model family.

What the Guarantee Says

- Majority of evaluated models show **consistency scores below 70%**: over 30% of instances produce different correctness outcomes depending on whether evidence is provided.
- Grounding precision (temporal IoU, spatial IoU) remains **low across all model families**, including proprietary ones.
- Open-source models **trail proprietary by 15–20 pp** on the VL track; the gap widens on the AV track.
- Models that score well on VL tasks **do not uniformly transfer** to AV tasks, showing audio-visual reasoning requires more than visual competence.

Experimental Findings

- **Models (50 total):** Proprietary: GPT-4o, Claude Opus, Gemini models. Open-source: Qwen-VL, LLaVA variants, InternVL.
- **VL accuracy gap:** Proprietary average 15–20 pp above open-source on QA subtasks; smaller gap on perception, larger on reasoning.
- **AV accuracy gap:** The proprietary-to-open-source gap widens; several open-source models perform near chance on audio-visual causal reasoning.
- **Consistency:** Majority of models below 70%; temporal reasoning and causal reasoning show the lowest consistency.
- **Grounding:** T-IoU and spatial IoU are low across all families, confirming grounding—not just QA—is the bottleneck.

Ablations and Interpretation

Paired-gap analysis: Models with the smallest text-vs-multimodal accuracy gap tend to be the most accurate overall; genuine grounding correlates with benchmark performance.

Category breakdown: Temporal reasoning and causal reasoning show the lowest consistency scores, suggesting models rely heavily on surface-level linguistic cues for these harder categories.

Audio-visual transfer: Models that excel at VL tasks do not uniformly extend their advantage to AV tasks; audio-visual reasoning requires explicitly joint processing that visual-only training does not provide.

Adversarial filtering: Questions solvable from text alone inflate apparent model capability; after filtering, accuracy on the remaining paired set drops substantially, exposing true grounding rates.

Annotation quality: All task categories meet inter-annotator agreement $\kappa > 0.8$; the consistency-score diagnostic is robust to annotation noise because it compares a model against itself across conditions, not against potentially noisy labels.

Reference

Xuyang Liu, Shihao Wang, Xinyu Wei, Minghan Li, Zeqing Wang, Zuhao Yang, Zichen Wen, Lei Zhang. **OmniCompass: A Holistic Benchmark for Omni-Modal Video Grounding and Understanding**. arXiv:2610.18829, October 2026.

https://arxiv.org/abs/2610.18829

Keywords: reinforcement learning • self-improvement • omni-modal • RL scaling

MiMo-V2.6: Scaling Reinforcement Learning Towards Self-Improvement

Scaling RL Compute Rather Than Model Size Raises DeepSWE Score from 58.4 to 72.6 Over $2.6M of Training, With Gains Still Accumulating at the Training Cutoff

By Xiaomi LLM-Core Team

arXiv:2610.11959 • Technical Report • October 2026

MiMo-V2.6 demonstrates that scaling *reinforcement learning compute*—rather than model parameters—on an omni-modal foundation yields sustained capability growth across code, reasoning, visual, and cybersecurity domains. The flagship MiMo-V2.6-Pro model raises its DeepSWE score from 58.4 to 72.6 over $2.6 M of RL compute, with the capability curve still rising at the training cutoff, suggesting the self-improvement regime has not saturated.

AT A GLANCE

Problem: Post-training RL runs for a fixed budget; it is unknown whether continued RL investment yields sustained gains or quickly plateaus.

Why it matters: If RL self-improvement does not saturate, the inference cost frontier can be pushed further with compute rather than new data collection.

Method: Mid-training broadens the exploration space; async RL at 1M context with frozen MoE router and a groupwise agentic grader prevents reward hacking.

Result: DeepSWE 58.4 → 72.6 over $2.6M RL compute; curve still rising at cutoff.

Evidence: DeepSWE, AutomationBench, MiMo Visual Coding, MiMo Cyber Bench.

The Big Picture

Pretraining scaling laws predict how capability grows with data and parameters, but they say little about the post-training phase. Reinforcement learning from verifiable rewards (code correctness, math check, search outcome) has become the dominant technique for pushing reasoning beyond supervised baselines, yet practitioners stop RL when a validation metric stalls—without knowing whether the curve has truly saturated or merely slowed.

MiMo-V2.6 provides the first systematic study of RL compute scaling on an omni-modal model: by tracking capability as a function of cumulative RL cost (in dollars) across four domains, the authors find super-linear returns up to at least $2.6 M.

Why Existing Approaches Fall Short

SFT scaling is bounded by annotation quality and coverage; it cannot discover solutions beyond what human demonstrators provide.

Short RL runs leave capability on the table. Prior work reports that accuracy curves are still rising at the training cutoff, but does not characterize the rate or shape of the growth.

Reward hacking is the primary failure mode in extended RL: agents discover shortcuts (swallowed exceptions, hard-coded test outputs, artificially long solutions) that inflate benchmark scores without improving genuine capability.

Modality siloing trains RL on a single modality; cross-modal synergies between code, visual, and reasoning capabilities remain unexploited.

Core Mechanism

Four components work together:

Mid-training expansion: Before RL begins, the model is trained on a broad multimodal corpus to widen its parameter distribution, providing more local optima for RL exploration to discover.

Asynchronous RL at scale: Rollout workers generate episodes concurrently; a central policy-update server applies gradient steps every N rollouts. The MoE router is frozen to prevent expert collapse.

Groupwise agentic grader: Solutions are evaluated in groups. The grader applies penalties for (a) reward hacking (swallowed exceptions, `try/except` catch-all, hard-coded outputs) and (b) unnecessary verbosity beyond the per-task length budget.

Multi-domain environments: Training spans code (software engineering), general reasoning (math, science), visual tasks (image understanding, chart reasoning), and cybersecurity (CTF, vulnerability analysis).

Technical Formulation

Let π_θ be the policy, $\mathcal{E} = \{e_1, \ldots, e_K\}$ the RL environment suite, and R_{grader} the grader reward. The training objective is:

$$\mathcal{J}(\theta) = \sum_{k=1}^{K} w_k \, \mathbb{E}_{\tau \sim \pi_\theta,\, e_k}[R_{\text{grader}}(\tau)]$$

where w_k are per-domain weights. The grader shapes the reward as:

$$R_{\text{grader}}(\tau) = R_{\text{base}}(\tau) - \lambda_h H(\tau) - \lambda_l L(\tau)$$

with $H(\tau) \in [0,1]$ the heuristically detected hack score, $L(\tau)$ the excess-length penalty, and λ_h, λ_l tuned constants.

Capability is measured by a benchmark score $B(\cdot)$ as a function of cumulative RL cost C. Empirically:

$$B(\text{DeepSWE}; C) = 58.4 + f(C), \qquad f(C) \approx \alpha C^{\beta} \quad (\beta > 1)$$

indicating super-linear (power-law) returns over the measured range up to $C = \$2.6\text{M}$.

Learning or Inference Procedure

1. **Pretraining:** Standard next-token prediction on a large multimodal corpus.
2. **Mid-training:** Continued training on a broad-domain curated dataset to expand exploration space.
3. **SFT warmup:** Short supervised phase initializes the policy in the target task distribution.
4. **Async RL:** Rollout workers generate episodes; the central server updates policy every N steps with the frozen MoE router.
5. **Grader-in-the-loop:** After each rollout batch, the groupwise agentic grader assigns shaped scalar rewards.
6. **Evaluation:** Periodic checkpoints are evaluated on DeepSWE, AutomationBench, MiMo Visual Coding, and MiMo Cyber Bench.

What the Guarantee Says

The paper does not provide formal convergence guarantees, but it offers an empirical finding that functions as an operational guarantee: the capability curve follows a super-linear growth law in RL spend across at least the $[0, \$2.6\text{M}]$ range, with no sign of saturation at the training cutoff. This distinguishes MiMo-V2.6 from reports where RL gains taper quickly.

The groupwise agentic grader provides an anti-hacking guarantee: solutions classified as hacked receive reward $R_{\text{grader}} \leq R_{\text{base}} - \lambda_h$, ensuring that the gradient signal penalizes shortcutting behaviors.

Experimental Findings

- **DeepSWE:** MiMo-V2.6-Pro rises from 58.4 → 72.6 over \$2.6 M RL; curve still climbing at cutoff.
- **AutomationBench:** Consistent positive trajectory across RL training.
- **MiMo Visual Coding:** Steady gains in visual task variants.
- **MiMo Cyber Bench:** Improvements in CTF and vulnerability tasks.
- **Reward hacking:** Grader reduces hack rate on held-out evaluation; without grader, hack rate monotonically increases with RL compute.
- **Open release:** Training dynamics, RL environments, and RL framework are released publicly.

Ablations and Interpretation

Without mid-training: RL exploration collapses faster; the policy finds fewer novel strategies; final benchmark score is lower.

Unfrozen MoE router: Expert load collapses; some experts receive nearly all routing weight, degrading general capability across modalities.

Without agentic grader: Reward hacking (swallowed exceptions, artificially long solutions) increases with RL time; benchmark-reported scores inflate while genuine held-out capability stagnates.

RL compute scaling curve: A log-linear fit underestimates measured growth; a power-law (super-linear) fit describes the data better across the full measured range.

Why self-improvement persists: Mid-training creates a rich parameter-space landscape with many local optima better than the SFT warmup. RL exploration finds progressively better optima as compute increases, consistent with a smooth power-law rather than a sharp saturation cliff.

Reference

Xiaomi LLM-Core Team. **MiMo-V2.6: Scaling Reinforcement Learning Towards Self-Improvement**. Technical Report. arXiv:2610.11959, October 2026. https://arxiv.org/abs/2610.11959